



Università
degli Studi di
Messina

X3DH protocol

Department:

Scienze matematiche e informatiche, scienze fisiche e scienze
della terra

Study Course:

Data Analysis

System Security

Student:

Adem Ammar (547572)

Professor:

M.villari/A.Ruggeri

Table of Contents

I.	Introduction and Project Idea:.....	3
1.	Project overview:	3
2.	Toit language:	3
II.	System architecture and Technologies:.....	4
1.	Architecture overview:	4
2.	Languages and technologies:	4
3.	Architectural constraints and adaptations:	5
4.	Design Trade-offs:.....	5
III.	Project Structure:	6
1.	Server-Side Structure:.....	6
2.	Client-Side Structure:	7
IV.	Cryptographic mechanisms:	7
1.	Cryptographic library and design philosophy:.....	7
2.	Key types and roles:	7
3.	X3DH implementation:.....	8
A.	Overview:	8
B.	DH computations:	8
C.	DH contributions:	9
D.	KDF:	9
4.	Authenticated encryption: (AEAD)	10
5.	Customized symmetric ratchet:	10
A.	Session state and directional chain keys:.....	10
B.	Message key derivation and chain advancement:.....	11
C.	Nonce derivation:	11
D.	Associated data (AD) binding:.....	11
6.	Vault encryption:	12
A.	Motivation:	12
B.	Passphrase-to-key derivation:	12
C.	Vault encryption using XChaCha20-Poly1305 + AD:.....	12
V.	UML Diagrams:.....	13
1.	X3DH session bootstrap:	13
2.	Established session:	14
3.	Key evolution:	15
4.	Vault decryption:	16

I. Introduction and Project Idea:

1. Project overview:

Modern secure messaging systems rely on **asymmetric cryptography** to establish shared secrets between users who may not be online at the same time.

A foundational primitive for this purpose is **Diffie–Hellman (DH)** key exchange, which allows two parties to derive a shared secret over an insecure channel without prior key sharing.

However, classic DH assumes both parties are online simultaneously. To address asynchronous communication, modern protocols such as **X3DH (Extended Triple Diffie–Hellman)** were introduced and are used in real-world systems like Signal.

X3DH enables a user to establish a secure session with another user who is offline by combining:

- long-term identity keys,
- signed prekeys,
- and one-time prekeys.

This project implements a **simplified** but security-correct messaging system based on X3DH.

The goal is not to replicate a production system, but to **demonstrate and implement the core cryptographic mechanisms** behind modern end-to-end encrypted messaging, including:

- authenticated key agreement,
- forward secrecy,
- offline message delivery,
- and secure key storage.

2. Toit language:

Toit is an open-source, object-oriented programming language specifically designed for the Internet of Things (IoT), particularly for microcontrollers like the **ESP32**. It basically aims to bring a high-level, efficient, and safe programming experience to embedded devices, offering an easier alternative to classic languages. In our project, we will be using Toit to implement our protocol, but without the actual microcontrollers; we will be running the code on the host machine.

II. System architecture and Technologies:

1. Architecture overview:

The system follows a **client-server architecture**:

- The **server** acts as:
 - a public key directory
 - a mailbox service for offline messages
 - and a relay for cryptographic operations
- The **client** is responsible for:
 - generating cryptographic keys
 - executing key agreement
 - encrypting and decrypting messages
 - and securely storing private keys

The server is intentionally designed to be **unable to decrypt messages**, ensuring end-to-end confidentiality.

2. Languages and technologies:

- **Toit**
Used for implementing the client-side logic.
Toit was chosen due to its constrained environment and limited standard library, which required explicit handling of cryptographic workflows.
- **Python (Flask)**
Used to implement the server and expose HTTP APIs.
- **C (libsodium)**
Used to implement cryptographic primitives (Diffie-Hellman, signatures, hashing, AEAD, Argon2id) exposed to the server through a native library.

>>> This separation allows cryptographic operations to be implemented securely while keeping client logic simple.

3. Architectural constraints and adaptations:

A significant challenge of this project was the **limited cryptographic and utility libraries available in Toit**.

In particular, Toit lacks:

- native support for advanced cryptographic primitives,
- password-based key derivation functions,
- flexible byte manipulation utilities.

To address these constraints, the following design choices were made:

- **All cryptographic primitives are implemented in a native C library** using “libsodium”.
- The server exposes cryptographic operations through HTTP endpoints.
- Clients act as **orchestrators**, requesting cryptographic operations rather than implementing them directly.

>>> This approach preserves correctness of the project guidelines while adapting to the limitations of the language.

4. Design Trade-offs:

Due to the chosen environment and project scope, several deliberate limitations were introduced:

- The protocol implements a **simplified symmetric ratchet**, not the full Signal Double Ratchet.
- Passphrases are provided via command-line arguments for simplicity.
- Cryptographic operations rely on server-mediated calls rather than local execution.

>>> These trade-offs were accepted to maintain clarity, debuggability, and feasibility within the constraints of the tools.

III. Project Structure:

1. Server-Side Structure:

The server is implemented using Python and follows a modular structure separating networking, storage, and cryptographic operations.

The server maintains:

- a directory of public key bundles,
- a mailbox queue per user,
- and a set of cryptographic endpoints exposed via HTTP.

Remark: Private keys are **never** stored on the server.

Public bundles include:

- identity public keys,
- signed prekeys,
- and one-time prekeys.

Mailbox entries contain **only encrypted messages**.

Example: (a registered user “mac” uploaded the bundle key into the server)

```
"users": {
  "mac": {
    "bundle": {
      "ik_dh_pub": "4RUTp5DI6UJJZILNFODBGryDt7Sy9m1uayTHd+iUQ1Cg=",
      "ik_sign_pub": "yfXPvD2BUI1CqD1Itq9UhPu5nR9LGTQhQY8WrMU7DyE=",
      "spk_id": 1,
      "spk_pub": "i5qxCJUw1hP6vK46hGZvhc2GwnMCbfK2d9d+z9Xm6Gg=",
      "spk_sig": "vZw2ei9ZHp2H3bukVd4dKBFk+ayw4taP1F9gmXJJr69+6Mm1ZVwafnDWhyHjOKi+SbdbKtz0R2UdZaK0Fbf4Cw==",
      "opks": [
        {
          "id": 1,
          "pub": "H5mryPl0t3RE0jOKS+zAAPoNKQReOq4Un2mkqUeTWEY=",
          "used": true
        },
        {
          "id": 2,
          "pub": "+8ne1GfmQ9w7KIL7iWvQOXae7B1X7EOhiUEo+mZuejA=",
          "used": false
        },
        {
          "id": 3,
          "pub": "xS5CgVKBdKkT46PXumNhbtCY3bdOustEKcYrJjtW8no=",
          "used": false
        },
        {
          "id": 4,
          "pub": "I7nuKWddZxsgjP9BPU94dMKWhyHl1TzClu8LZLa1DSg=",
          "used": false
        },
        {
          "id": 5,
          "pub": "VEXxdr3gZNm1usihsIgeCMZomZ7SoKwa35c+wuFFjGg=",
          "used": false
        }
      ]
    }
  }
}
```

2. Client-Side Structure:

Each client (user) maintains a local state directory:

```
state/<user>
|-----secrets.enc
|-----secrets_meta.json
|-----public.json
|-----sessions.json
```

Where:

- “secrets.enc”: Encrypted vault containing all private keys
- “secrets_meta.json”: Stores vault metadata (salt, nonce)
- “public.json”: all of public keys used for bundle publication
- “sessions.json”: Stores **per-peer** session state, including chain keys and counters.

>>> This structure ensures **separation of concerns** between long-term identity material, session state, and public information.

IV. Cryptographic mechanisms:

1. Cryptographic library and design philosophy:

The project implements an end-to-end encrypted messaging workflow in which two users can establish a shared session key even when one party is **offline** and subsequently exchange confidential and integrity-protected messages.

The protocol is organized into two phases: (i) **session bootstrap** using **X3DH** (Extended Triple Diffie-Hellman) and (ii) **session messaging** using a lightweight symmetric ratchet that derives a fresh **message key** for each ciphertext. A minimal server is assumed to provide availability services namely distributing public key bundles and storing mailbox messages but the server is treated as **untrusted** with respect to message confidentiality and integrity. The design explicitly separates long-term identity material (stored on the client) from per-session keys and per-message keys, limiting the blast radius of compromise.

2. Key types and roles:

Each user maintains multiple key pairs with distinct lifetimes and security roles. A **long-term identity key** is used to bind a user’s identity to cryptographic material. In the implementation, X3DH uses **X25519** for Diffie–Hellman key

agreement, while **Ed25519** is used for digital signatures. A **signed prekey (SPK)** is generated and then signed using the Ed25519 identity signing key, creating a verifiable binding between the user identity and a medium-term Diffie–Hellman public key. Additionally, the user generates a set of **one-time prekeys (OPKs)**, intended to be consumed during initial session establishment to provide stronger forward secrecy. Finally, whenever a session is initiated, the initiator generates an **ephemeral key (EK)** (X25519) to guarantee freshness and prevent reuse of initial shared secrets.

- ✓ This separation of key roles is important: identity keys authenticate who you are, signed prekeys enable offline session setup while remaining authenticated, and one-time prekeys reduce the damage of a later compromise by ensuring the initial secret is unique per session attempt.

3. X3DH implementation:

A. Overview:

The project implements the **Extended Triple Diffie–Hellman (X3DH)** handshake to establish a shared secret between an initiator (Alice) and a recipient (Bob) in an asynchronous setting. The core idea is to compute multiple Diffie–Hellman values using a combination of long-term and ephemeral keys, and then compress them into a single session secret via a KDF. In our code, the X3DH shared secret is computed using up to **four** X25519 DH operations and then hashed using libsodium’s “crypto_generichash”

B. DH computations:

Our functions explicitly define the four DH computations on each side:

Alice derives (the initiator):

- $DH1 = DH(A_ik_priv, B_spk_pub)$
- $DH2 = DH(A_ek_priv, B_ik_pub)$
- $DH3 = DH(A_ek_priv, B_spk_pub)$
- $DH4 = DH(A_ek_priv, B_opk_pub)$ (only if an OPK is available)

Bob derives (the recipient):

- $DH1 = DH(B_spk_priv, A_ik_pub)$
- $DH2 = DH(B_ik_priv, A_ek_pub)$
- $DH3 = DH(B_spk_priv, A_ek_pub)$
- $DH4 = DH(B_opk_priv, A_ek_pub)$ (only if OPK was used)

- **Note:** These pairings are arranged so that both sides compute the **same** byte strings in the same order, even though they use different private/public inputs.

C. DH contributions:

Each DH term contributes a specific security goal; the protocol intentionally mixes static and ephemeral components:

- **DH1 (IK × SPK):**

DH1 binds the initiator's identity key to the recipient's signed pre-key. Since Bob's SPK is signed with Bob's Ed25519 identity signing key, Alice's verification step ensures she is using Bob's authentic SPK. This makes it substantially harder for a malicious server to mount a key-substitution attack without detection, because the handshake key depends on a signed public key controlled by Bob.

- **DH2 (EK × IK):**

DH2 ensures the handshake depends on Alice's freshly generated ephemeral key and Bob's long-term identity key. This contributes freshness (new EK per session) while still binding the handshake to Bob's identity. Even if DH1 were reused (e.g., SPK reuse over a period), DH2 prevents the final secret from being fully repeatable because it includes Alice's new EK.

- **DH3 (EK × SPK):**

DH3 mixes Alice's ephemeral key with Bob's signed pre-key, strengthening forward secrecy properties relative to relying solely on DH1/DH2. Practically, it ensures that compromise of a single long-term key is less likely to be sufficient to reconstruct the handshake secret.

- **DH4 (EK × OPK):**

DH4 is computed only when a one-time pre-key is available. Because OPKs are intended to be consumed, DH4 gives each initial session a unique secret contribution that is not reused across sessions. This is the component that most directly improves forward secrecy in the asynchronous setting: even if Bob's signed pre-key persists for some time, the one-time pre-key prevents the initial shared secret from repeating.

D. KDF:

After computing DH1>>DH4, the code concatenates them in order and derives a **32-byte secret** using libsodium's "crypto_generichash" with a fixed prefix:

- Input: "X3DH" || DH1 || DH2 || DH3 || (DH4 if present)

- Output: SK

This is implemented as `x3dh_kdf()` in `crypto.c`. The "X3DH" prefix provides [domain separation](#), preventing accidental cross-protocol key reuse if similar DH material were ever used elsewhere.

4. Authenticated encryption: (AEAD)

For message confidentiality and integrity, our wrapper in C uses XChaCha20-Poly1305 “`crypto_aead_xchacha20poly1305_ietf_encrypt/decrypt`”

This construction provides two critical properties simultaneously: (i) [confidentiality](#) (ciphertexts do not reveal plaintext) and (ii) [integrity/authenticity](#) (modification is detected during decryption). XChaCha20-Poly1305 is particularly suitable for application protocols because it uses a 24-byte nonce and has strong misuse-resistance characteristics compared to older designs.

Importantly, our implementation uses Associated Data (AD) to bind metadata to the ciphertext. If an attacker changes fields such as sender/receiver identifiers or counters, the AEAD verification fails, preventing silent tampering.

5. Customized symmetric ratchet:

A. Session state and directional chain keys:

After X3DH derives the initial session secret SK, our design transitions into a symmetric “ratchet-like” mechanism based on evolving **chain keys**. Instead of a full Double Ratchet (which also ratchets Diffie–Hellman over time), we implement a purely symmetric ratchet that advances a [per-direction](#) chain key [every time](#) a message is sent or received.

A key design choice is that the session maintains *two independent chain keys*:

- [ck_send](#): used to derive message keys for messages sent from user → peer
- [ck_rcv](#): used to derive message keys for messages received from peer → user

This separation ensures that compromise of one direction’s state does not automatically reveal the other direction’s message keys, and it also prevents key/nonce reuse across directions.

Our Toit code derives the two chain keys from SK using a hash-based KDF structure that incorporates directionality and user identities:

- $ck_send = H("CKS|" + ctx + "|" || SK)$
- $ck_rcv = H("CKR|" + ctx + "|" || SK)$

- where ctx = "from->to" (a contextual string binding the direction to identities)
 - This construction accomplishes two things: it provides domain separation between sending and receiving keys, and it binds chain keys to the sender/receiver identities, reducing the risk that the same SK is interpreted identically in different contexts.

B. Message key derivation and chain advancement:

For each message, a fresh **message key (MK)** is derived from the current chain key, and then the chain key is advanced. In our implementation:

- $mk = H("MK" || ck)$
- $ck_next = H("CK" || ck)$

This design ensures that even if an attacker learns a past message key, it does not trivially yield future message keys because the chain **is one-way** (hash-based). Conversely, if an attacker compromises the current chain key, they can derive *future* keys (a limitation of symmetric-only ratchets), but they cannot derive *past* chain keys due to preimage resistance of the hash.

C. Nonce derivation:

Since XChaCha20-Poly1305 requires a 24-byte nonce, our code derives the nonce deterministically from (chain_key, counter):

- $h = H("NONCE" || ck || "|" || str(counter))$
- $nonce = \text{first_24_bytes}(h)$

The counter is maintained in session state (e.g., send_counter) and incremented per message. This construction is meant to guarantee nonce uniqueness per key evolution and message index.

D. Associated data (AD) binding:

We basically try to define two AD formats:

- **Normal messages:** from|to|init|counter
- **Initial messages:** includes additional fields: ik, ek, spk_id, opk_id

By placing these fields into AEAD associated data, our implementation ensures that ciphertexts are cryptographically bound to the intended metadata. An attacker who replays a ciphertext while changing "from/to" or the counter will cause AEAD verification **failure**, preventing undetected message relabeling.

6. Vault encryption:

A. Motivation:

A key part of real messaging systems is protecting long-term secrets on disk. In our design, a local “vault” stores private key material (identity DH private key, identity signing private key, signed prekey private keys, and one-time prekey private keys).

Storing these in plaintext would make a simple file exfiltration possible. To mitigate this, our implementation encrypts the vault with a user passphrase and stores only ciphertext + metadata.

B. Passphrase-to-key derivation:

My C wrapper exposes a function “pwhash_argon2id_32” that derives a 32-byte key using **Argon2id** with libsodium’s *moderate* parameters. A random 16-byte salt is generated for each vault encryption so that identical passphrases do not produce identical keys across users or re-encryptions.

C. Vault encryption using XChaCha20-Poly1305 + AD:

To encrypt the vault, the client:

1. Generates salt (16 bytes) and nonce (24 bytes).
2. Derives key = Argon2id(passphrase, salt) (32 bytes).
3. Encrypts the JSON-encoded vault using XChaCha20-Poly1305 with **Associated Data** set to a vault context string:
AD = "vault-v1|<user>"

This AD binds the vault ciphertext to the specific username and vault version. If someone swaps ciphertext files across users, decryption will fail because the AD no longer matches, preventing a class of mix-and-match attacks.

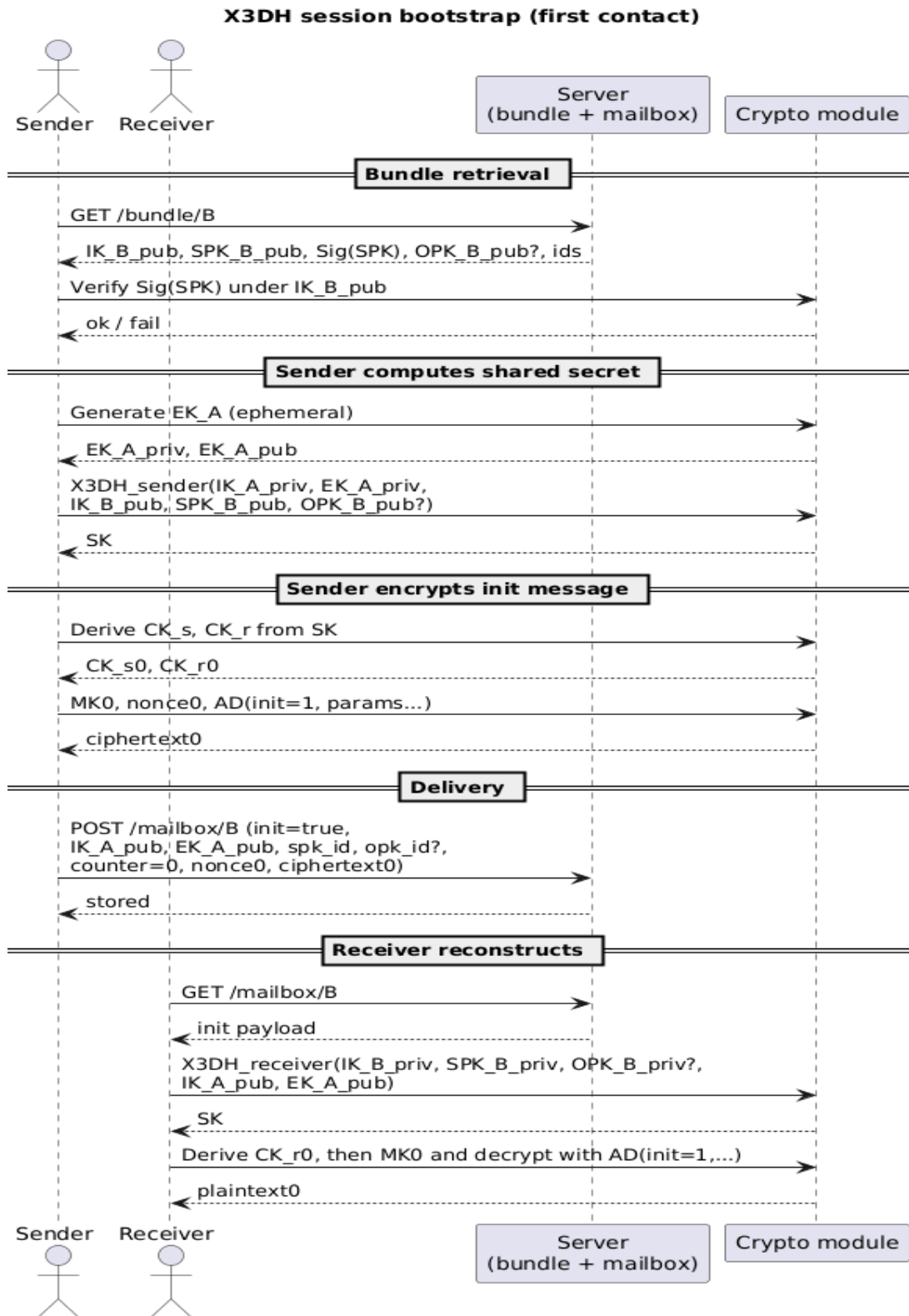
The vault is stored as:

- secrets.enc (ciphertext, base64)
- secrets_meta.json containing salt_b64 and nonce_b64

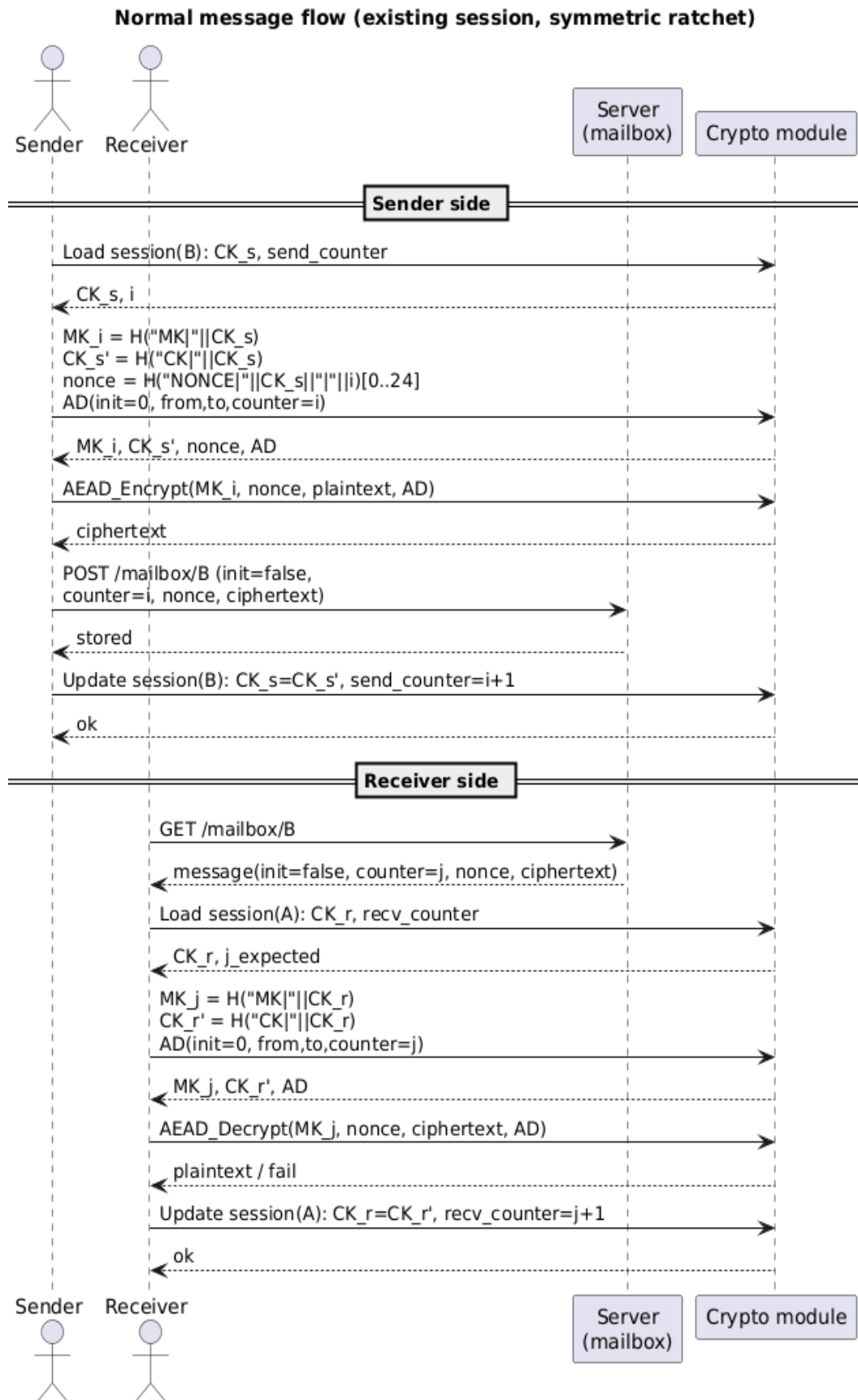
On load, the client re-derives the key from the passphrase and salt, then attempts AEAD decryption; an incorrect passphrase fails cleanly because AEAD authentication fails.

V. UML Diagrams:

1. X3DH session bootstrap:

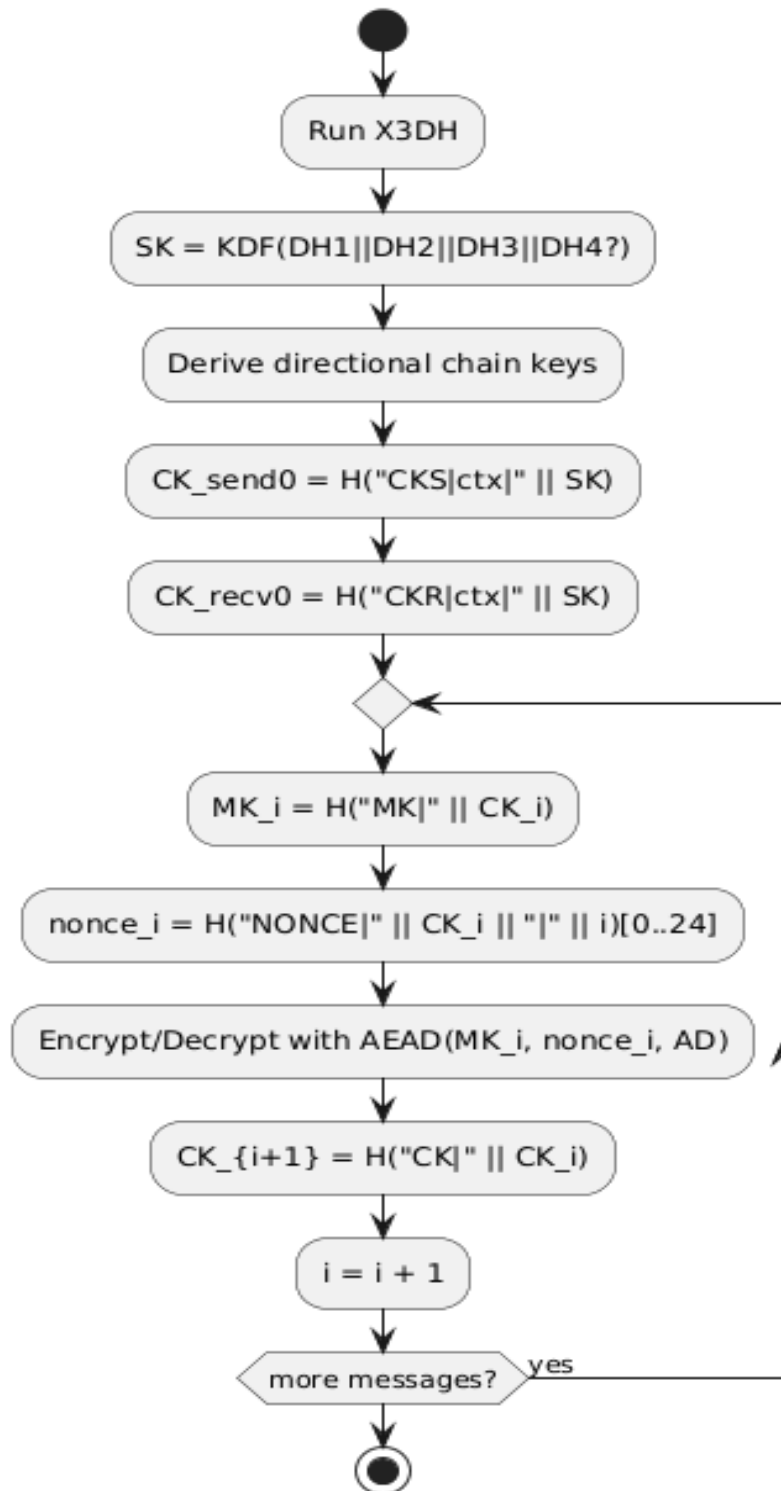


2. Established session:



3. Key evolution:

Key hierarchy: from X3DH to per-message keys



4. Vault decryption:

Encrypted vault: decrypting long-term private keys

